

Graph Neural Networks for Boundary Detection in Hardware Netlist Reverse Engineering

Anjali Sarawgi



Institute of Statistics, LMU Munich

July 16, 2026

- 1 Introduction
- 2 Dataset Overview
- 3 Methodology & Experimental Setup
- 4 Results
- 5 Error Analysis
- 6 Discussion
- 7 Limitations & Future Work

- **Integrated circuits (ICs)** are everywhere, but the global IC supply chain is not fully trustworthy (hardware trojans, IP piracy)
- **Reverse engineering (RE)** of ICs aims at recovering a chip's structure (*subcircuits*) from its physical implementation
- Manual RE is a slow, and an expert dependent process

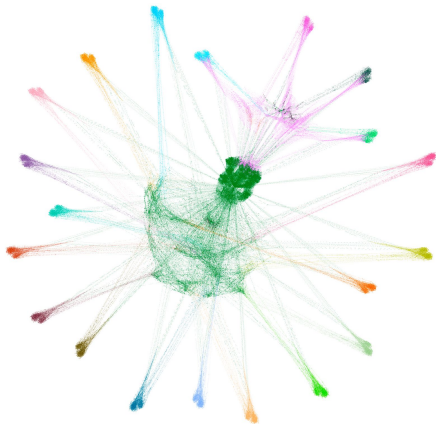


Figure: A circuit design: colors represent different subcircuits

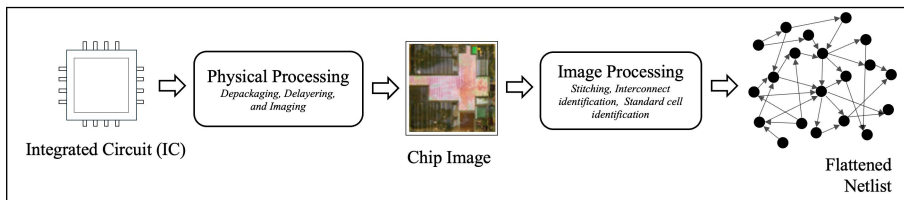


Figure: Reverse engineering process

- Flattening the design destroys all hierarchy → a **“sea of gates”**
- Large, dense, unstructured netlists
- No hierarchy, subcircuit information and boundaries

- **Prior works:** Explore subcircuit recovery from flattened netlists, but:
 - 1 Assumes subcircuit types are known during training
 - 2 Homogenous, curated datasets
 - 3 State that boundary nodes are distinct and cluster together
- **Our approach:** Rather than recovering full subcircuits directly, we focus on detecting where the subcircuits connect (*boundary nodes*) as an intermediate step:

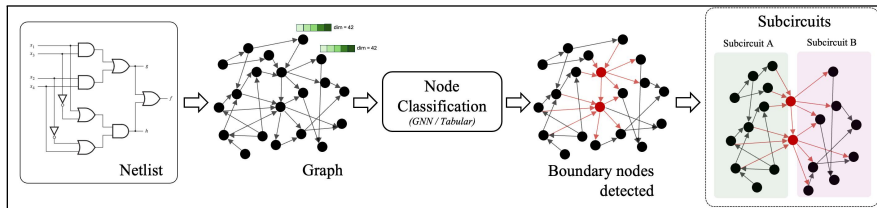
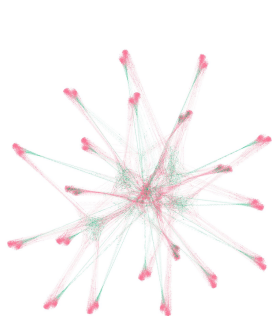


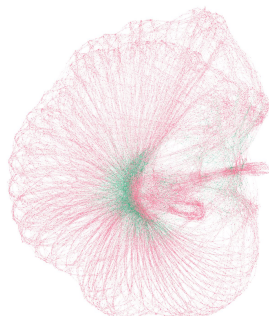
Figure: Boundary detection process

- **Research question:** Can *boundary nodes* be detected across heterogeneous designs purely from structural and signal flow characteristics, *without prior assumption about the design or subcircuit types*?
- **Task:**
 - Binary node classification
 - Boundary nodes = 1, Non-boundary nodes = 0
 - Nodes: gate types (eg. AND, OR), Edges = wires (directed)
- **Our pipeline:** Dataset → Experimental setup & Model selection
→ Results → Error Analysis → Discussion

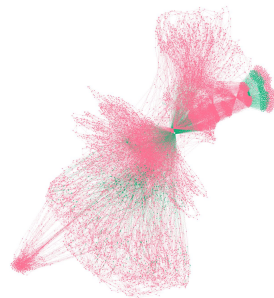
- 1 Introduction
- 2 Dataset Overview**
- 3 Methodology & Experimental Setup
- 4 Results
- 5 Error Analysis
- 6 Discussion
- 7 Limitations & Future Work



(a) aes_cipher_top



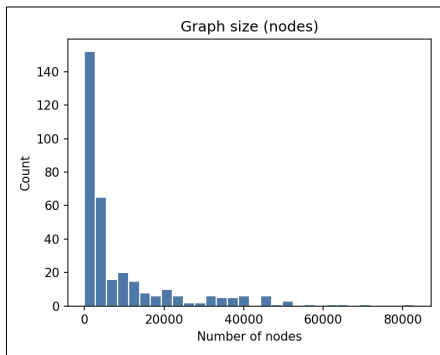
(b) ALU



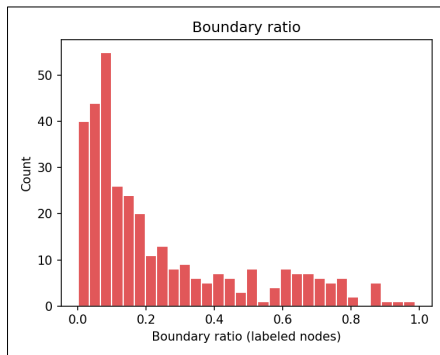
(c) sha256-core

Figure: Example of three gate-level netlist graphs. Green = boundary nodes, pink = non-boundary nodes.

- **Heterogeneity:** Different designs, with different logical functionalities (339 designs, 50 families, 3 libraries)



(a) Distribution of graph sizes (nodes)



(b) Distribution of boundary node ratios

- **Graph size:** Ranges from 12 to 82,947 nodes across designs (mean= 9,318, std. dev. = 13,943 nodes)
- **Boundary ratio:** Ranges from ~0% to 99% across designs

Design	Library	Subcircuits	Nodes	Top gates	BR
aes_cipher_top	gsc145nm	24	23,876	OR (50%) AND (21%) NOR (8%)	0.07
	nangate	24	13,717	NAND (29%) AOI (21%) NOR (16%)	0.14
	osu035	24	21,388	NAND (34%) NOR (20%) OAI (18%)	0.13

- **Library-induced structural differences:** Same design, same logic, same subcircuit count, *but* node count and boundary ratio can shift significantly across libraries
- Model must also generalize across all these differences

- 1 Introduction
- 2 Dataset Overview
- 3 Methodology & Experimental Setup**
- 4 Results
- 5 Error Analysis
- 6 Discussion
- 7 Limitations & Future Work

- Each node gets **42-dimensional feature vector**, aimed to capture both local properties and graph structural position
- Three categories:
 - **Gate type encodings (1–28):**
 - One-hot encoding of 14 logic gate types (AND, OR, XOR, ...)
 - 1-hop neighbour gate-type distribution
 - **Local neighbourhood features (29–31):**
 - In degree, out degree, fan-ratio
 - **Structural and positional features (32–42):**
 - Predecessor / successor degree mean & std
 - Forward reach, backward reach, reach asymmetry
 - Degree contrast, 2-hop ego density, k-core number, page rank

- **Evaluation Strategy:** Leave-one-design-out (LODO)
 - One design is entirely held out for each of the 50 families → Remaining families used for training
 - Test family is entirely unseen
- **Metric: PR-AUC**
 - Summarizes precision-recall curve as a single value
 - Reliable with class imbalance
 - Reflects the ranking ability of the model (threshold-free)
 - Baseline: Random classifier PR-AUC equals the boundary ratio

- ① **Tabular baselines:** Random Forest, and TabPFN (50,000 training nodes), TabICL (200,000 training nodes) (foundation models for tabular data)
 - Treats each node independently, no graph structure used
 - PR-AUC: 0.51, 0.52 and 0.50 respectively for AES family (all three perform similarly)
- ② **Graph Neural Networks (GNNs)**
 - Uses graph structure via message passing
 - Classifies nodes: *its nodes own features + its neighbors features*
 - PR-AUC: 0.57 for AES family

Note: *We use Random Forest as the tabular baseline going forward (speed, interpretability)*

ID	Feature	Imp.	ID	Feature	Imp.
38	PageRank	0.15	33	Predecessor deg. std	0.05
41	Reach asymmetry	0.09	27	DFF (one-hot)	0.04
40	Backward reach	0.06	21	XOR (percentage)	0.03
36	2-hop ego density	0.06	34	Successor deg. mean	0.03
32	Predecessor deg. mean	0.06	7	XOR (one-hot)	0.03
39	Forward reach	0.06	18	OR (percentage)	0.02
42	Degree contrast	0.05	31	Fan ratio	0.02
			29	In degree	0.02

Table: Top 15 most important features ranked by random forest.

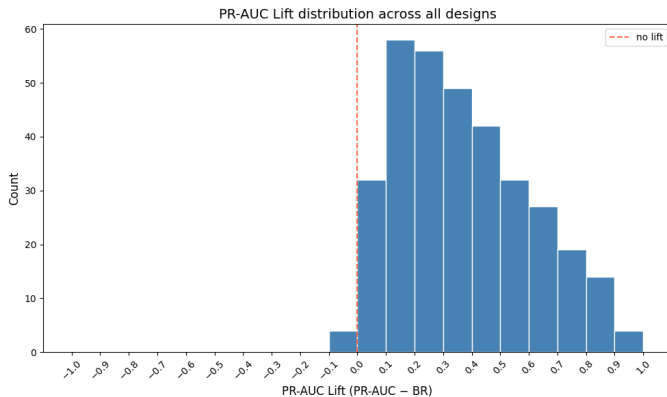
- Top 8 features are all structural features
- First gate-type features appear at position 9

- Final GNN model configuration after model selection:

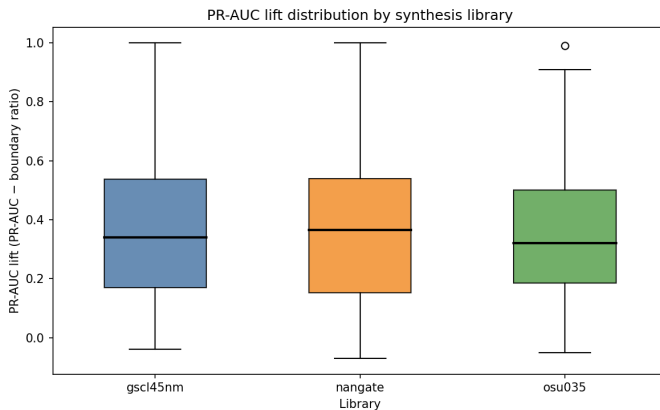
Parameter	Value	Parameter	Value
Architecture	Bi-GraphSAGE	Loss	Weighted CE
Aggregation	Mean	Optimizer	AdamW
Layers	6	LR	0.001
Hidden dim	256	Epochs	300 (w/ early stopping)

- GraphSAGE:** Inductive (generalizes to unseen graphs)
- Final model:** Bi-directed GraphSAGE (captures both forward and backward signal flow context)

- 1 Introduction
- 2 Dataset Overview
- 3 Methodology & Experimental Setup
- 4 Results**
- 5 Error Analysis
- 6 Discussion
- 7 Limitations & Future Work



- Mean PR-AUC lift = 0.37, Std. dev. = 0.24
- 237 of 339 (70%) designs achieve a PR-AUC lift of ≥ 0.20
- Results show high variance across designs



- Median: gsc145nm = 0.34, nangate = 0.37, osu035 = 0.32
- Performance is consistent across all three libraries; nangate highest

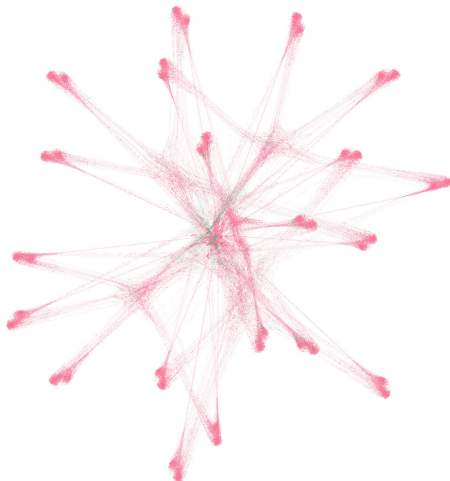
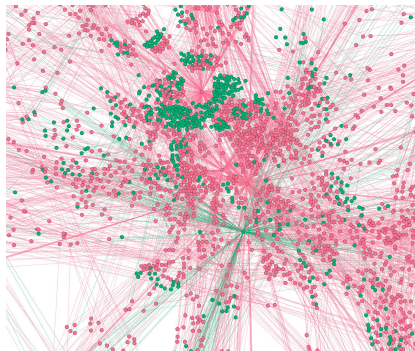


Figure: Design: aes_cipher_top (osu035). PR-AUC = 0.99, Baseline = 0.13.
Pink = correct predictions, Green = incorrect predictions.



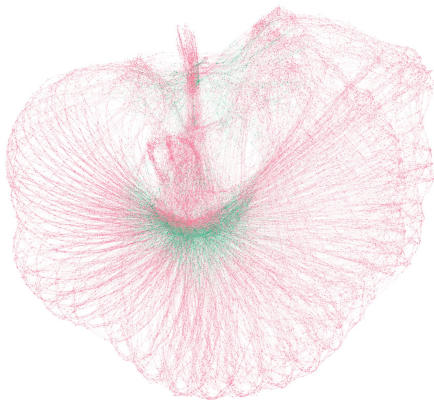
(a) Zoomed 1



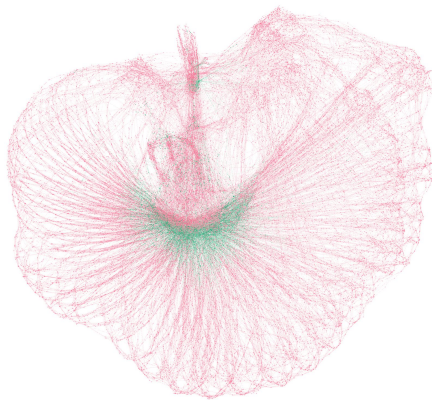
(b) Zoomed 2

Figure: Pink = correct predictions, Green = incorrect predictions.

- Errors concentrate in denser regions
- Performs well for Repeated and clear structures at periphery (sbox)

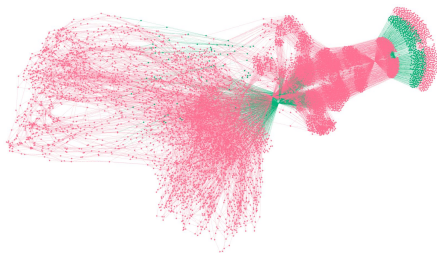


(a) Ground Truth

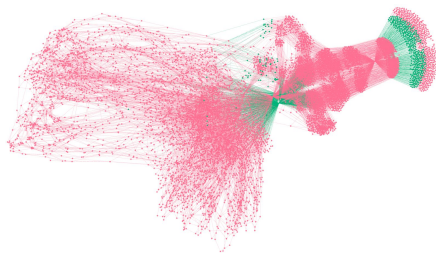


(b) Prediction

Figure: Design: ALU (osu035). PR-AUC = 0.87, Baseline = 0.07. Pink = non-boundary, Green = boundary nodes.

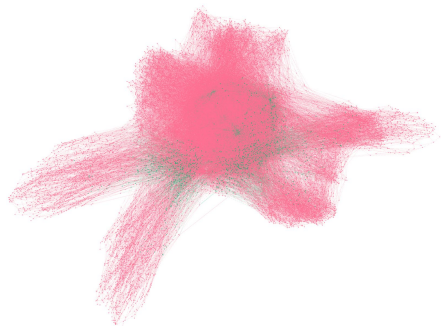


(a) Ground Truth

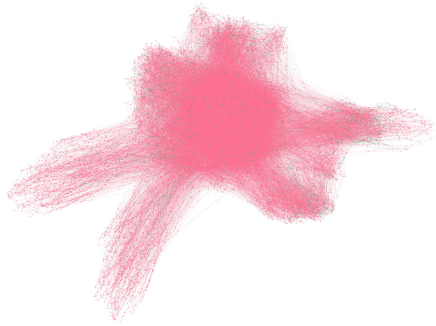


(b) Prediction

Figure: Design: sha1_core (osu035). PR-AUC = 0.95, Baseline = 0.07. Pink = non-boundary, Green = boundary nodes.

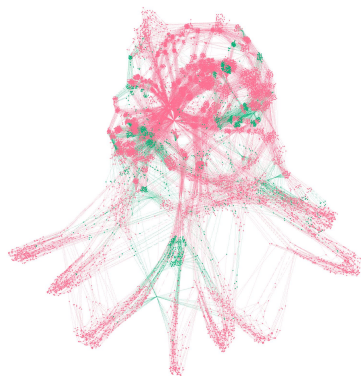


(a) Ground Truth

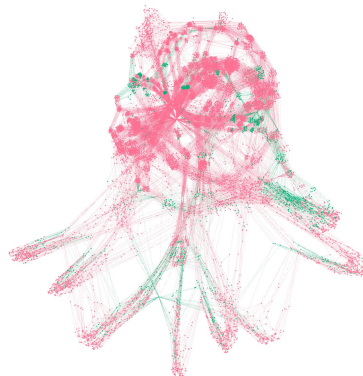


(b) Prediction

Figure: Design: chacha (osu035). PR-AUC = 0.06, Baselines = 0.04. Pink = non-boundary, Green = boundary nodes.



(a) Ground Truth



(b) Prediction

Figure: Design: apbtoaes128 (aes_core) (osu035). PR-AUC = 0.28, Baselines = 0.13. Pink = non-boundary, Green = boundary nodes.

- 1 Introduction
- 2 Dataset Overview
- 3 Methodology & Experimental Setup
- 4 Results
- 5 Error Analysis**
- 6 Discussion
- 7 Limitations & Future Work

- **Q1:** Do graph properties (size, density, imbalance) affect performance?
- **Q2:** Does graph structure affect performance?
- **Q3:** Is there structure in the errors, or do they occur randomly?
- **Q4:** Does model fail at graph level, or is it partition specific?
- **Q5:** Does model choice matter?

For analysis, we use: (i) Crypto family, osu035 library (45 designs), (ii) top-k classification, and (iii) PR-AUC lift ≥ 0.20 as model success.

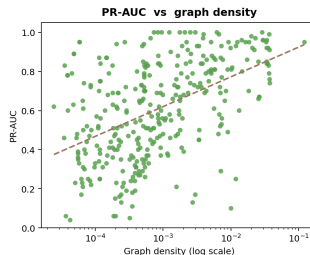
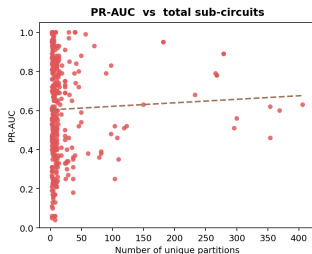
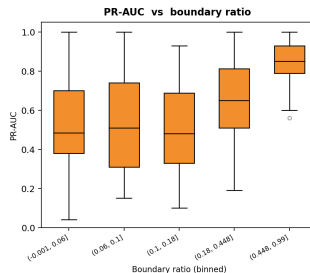
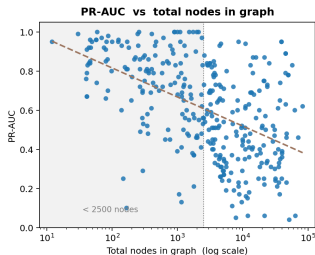
Q1: Do graph properties affect performance?

- **Graph size:** Smaller graphs perform consistently well, larger graphs show high variance
- **Graph density:** Moderate positive correlation with PR-AUC ($\rho = 0.50, p < 0.0001$) i.e. denser graphs support better boundary detection
- **Boundary ratio & partition:** No strong effect on performance
- **Large design generalization:**

Design	Nodes	BR	PR-AUC (GNN)	PR-AUC (RF)
tiny_aes	$\geq 300,000$	0.21	0.62	0.58

Note: Moderate negative correlation ($\rho = -0.52$) between graph size and boundary ratio.

Q1: Do graph properties affect performance?



Note: PR-AUC without lift, 339 designs

Q2: Does graph structure affect performance?

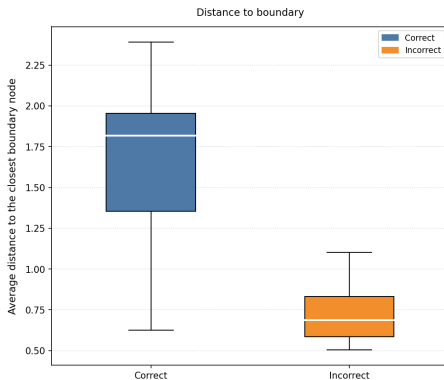
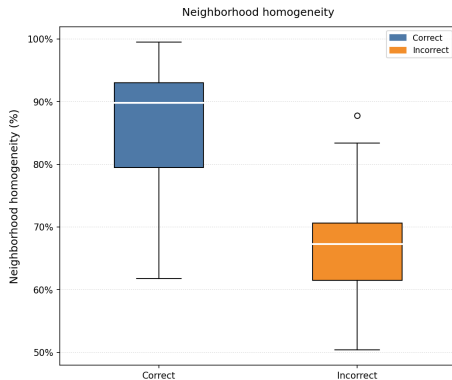
Yes: three structural conditions predict success or failure

- 1 **High Modularity score (Q):** a well-separated community structure
- 2 **Distinct Reach asymmetry:** boundary nodes balanced; internal nodes have higher backward-reach
- 3 **High PageRank:** boundary nodes are globally more important

Conditions	Failure rate	n
All three present	0% [0–20%]	15
All three absent	87.5% [53–98%]	8

Note: We have a small sample size ($n=45$: 32 success, 13 = failure).

Q3: Is there structure in the errors?



- 1 Errors occur in structurally mixed regions (median homogeneity%: 90% vs 67%)
- 2 Errors concentrate near true boundaries (median distance: 1.82 vs 0.69)

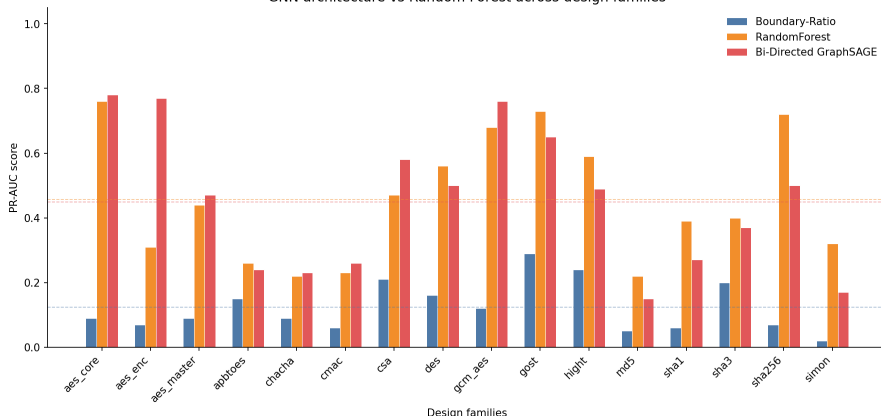
Q4: Is Failure Partition-Specific

Graph	Partition	Nodes	PR-AUC	Baseline
aes_key_expand_128	aes_key_expand_128	1,427	0.99	0.09
	aes_sbox	3,176	1.00	0.05
	aes_rcon	93	*1.00	0.53
aes_inv_cipher_top	aes_inv_sbox	13,587	0.10	0.04
	aes_inv_cipher_top	11,994	0.31	0.03
	aes_key_expand_128	1,957	0.62	0.26
	aes_sbox	2,758	0.99	0.15
	aes_rcon	69	*0.62	0.36

- * $n < 100$ nodes (small subcircuit), **Blue** = unseen subcircuit type during training
- Model does not fail uniformly across the graphs (matches intuition)

Q5: Does model choice matter?

GNN architecture vs Random Forest across design families



- Baseline = 0.12, Avg. PR-AUC: RF = 0.444, GNN = 0.468¹
- 21 RF, 20 GNN, and 4 as ties

¹For all 339 designs: GNN = 0.61, RF = 0.55

Feature ablation:

- **Random Forest:** drops sharply with fewer features
0.44 (42f) $\rightarrow$ 0.32 (31f) $\rightarrow$ 0.26 (14f)
- **GNN:** stable across 14, 31, 42 features
0.45 (42f) $\rightarrow$ 0.47 (31f) $\rightarrow$ 0.45 (14f)
- GNN learns from graph structure directly, and even improves with lesser features (peaks slightly at 31 features)

When to use which:

- GNN better in high-modularity graphs ($\rho = 0.55$, $p < 0.001$)²;
- Random Forest more stable in low-modularity settings ($\rho = 0.24$, $p = 0.11$)
- RF trains $\sim 4\times$ faster (3.85 vs 16.40 minutes)
- Neither model consistently dominates:
($W = 425.50$, $p = 0.948$) (Wilcoxon signed-rank test on PR-AUC)
- GNN may not always be the better choice

²Spearman correlations (ρ).

Design	BR	Nodes	0%	5%	10%	20%
aes_core	0.09	17,058	0.78	0.74	0.72	0.61
gost28147-89_latest	0.30	13,348	0.65	0.66	0.63	0.65
hight_latest	0.24	2,008	0.49	0.51	0.51	0.46
uart2spi_latest	0.06	560	0.95	0.84	0.82	0.69

Table: PR-AUC under graph noise applied at evaluation only.

- **Noise types:**

- ① Gate type noise
- ② Feature perturbation
- ③ Edge corruption

Graph	PageRank		2-hop ego		Reach Asym		Mod (Q)	PR-AUC	
	B	I	B	I	B	I		GNN	RF
osu035									
sha256_core	0.75	1.07	0.23	0.10	0.22	-0.26	0.71	0.92	0.87
sha256_stream	0.68	1.06	0.35	0.10	0.29	-0.26	0.72	0.66	0.69
gsc145nm									
sha256_core	0.72	1.09	0.03	0.11	-0.16	-0.28	0.66	0.23	0.60
sha256_stream	0.69	1.08	0.02	0.10	-0.22	-0.28	0.67	0.19	0.72

Table: Structural features and PR-AUC for sha256-master across two libraries. B = boundary nodes, I = internal nodes. **Pink** = PR-AUC lift < 0.20.

- Same design, different library → structural signals collapse → model performance changes.

- 1 Introduction
- 2 Dataset Overview
- 3 Methodology & Experimental Setup
- 4 Results
- 5 Error Analysis
- 6 Discussion**
- 7 Limitations & Future Work

- **Partially:** model can identify a signal that generalizes beyond the training set (validated through LODO setup + 70% success lift).
- **Matches intuition:** model success when boundary nodes occupy distinct positions (high modularity, distinct reach asymmetry, higher page rank scores)
- **When structural signals are weak:** model benefits from seen structures during training (modified LODO)

- Random forest with strong engineered features matches GNN performance
- GNN stays stable even with minimal features
- Failures are subcircuit specific; and errors are near boundaries
- The model remains robust to subcircuit count, size, class ratios, library type and added noise

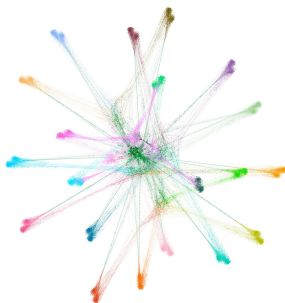
- Prior works assume subcircuit types are seen during training and tests only on seen subcircuit variants (homogenous datasets)
- We reformulate boundary detection as a **standalone structural task**, requiring no prior knowledge of the subcircuit type, and evaluate generalization
- Results support the boundary node hypothesis:
 - ① Meaningful lifts on completely unseen designs (70% of designs)
 - ② Errors not randomly spread (near true boundary regions)
- Lays foundation for a practical intermediate step toward subcircuit recovery in realistic reverse engineering scenario

- 1 Introduction
- 2 Dataset Overview
- 3 Methodology & Experimental Setup
- 4 Results
- 5 Error Analysis
- 6 Discussion
- 7 Limitations & Future Work**

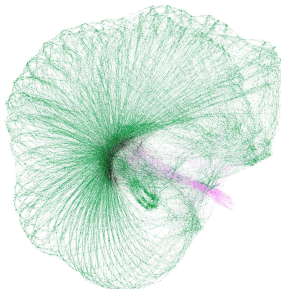
- **Metric:** PR-AUC measures ranking, not classification performance
- **Intermediate step:** Not an end-to-end solution for subcircuit functionality recovery
- **Wider evaluation:** Requires testing on more libraries, larger and more complex designs
- **Structural exposure:** Performance is not fully structure-independent; model still benefits from seeing structurally similar patterns during training (modified LODO)

- **Richer features:** Leveraging the logical structure; integrating functional information like *word-level encodings*, *bus-like patterns*, *positional encodings* and *signal probabilities*
- **Evaluation Metric:** Finding a practical way to predict boundary ratio without ground truth labels at test time
- **Structural predictors:** Identifying additional structural indicators which predict model success (also helps validate boundary node characteristics)
- **Evaluation dataset:** Building a standard benchmark dataset for strong reproducible evaluations and to support future work

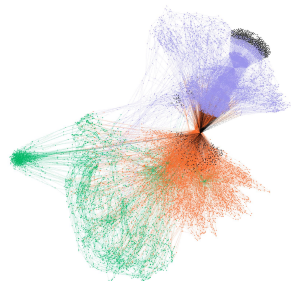
Thank you for your attention!



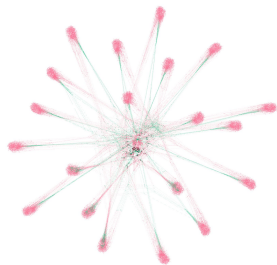
(a) Design: `aes-cipher-top`



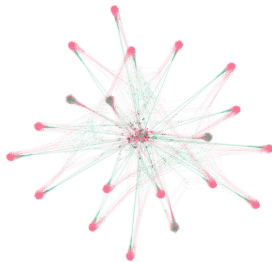
(b) Design: `ALU`



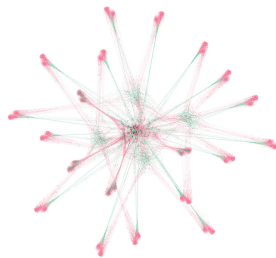
(c) Design: `sha256-master`



(a) gsc145nm



(b) nangate



(c) osu035

Figure: Pink nodes are non-boundary nodes ($\text{boundary} = 0$) and green nodes denote boundary nodes ($\text{boundary} = 1$). The black nodes are the @top nodes, which are masked as $\text{boundary} = 0$ for our study.

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

- Obtained by varying the classification threshold and plotting precision (y-axis) against recall (x-axis).
- Summarizes this curve as a single value
- Higher values (~ 1.00) indicate better performance

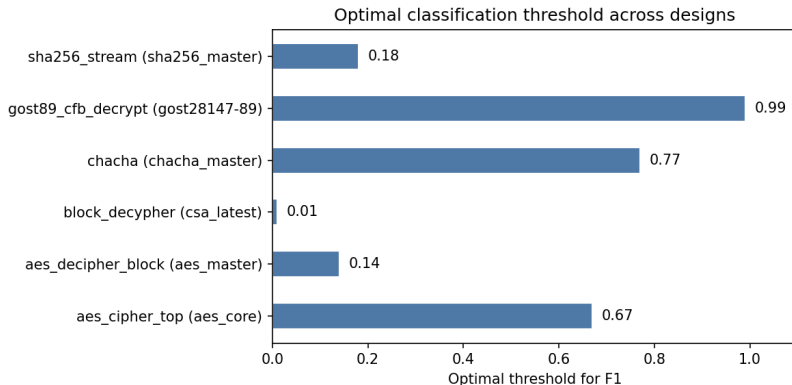
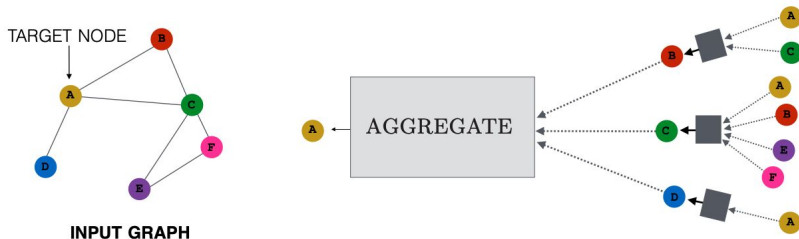


Figure: Bar graph showing optimal classification threshold across different graphs. The variations show that a single fixed threshold does not generalize well across designs.

Model	aes_core	aes_enc	aes_master	apbtoaes128	gcm_aes	Avg.
RF (all 42 features)	0.75	0.35	0.37	0.29	0.75	0.50
RF (top 15 features)	0.74	0.35	0.29	0.32	0.73	0.49

Table: Comparison of Random Forest performance using the full feature set versus the top 15 most important features. We use $n = 100$ estimators.



- A node is classified using **its own features + its neighbors features**
- At each layer, every node:
 - 1 **Aggregates** information from its neighbors
 - 2 **Updates** its own representation using that aggregated info
- After ℓ layers $\Rightarrow$ a node “sees” up to ℓ hops away
- **Risk:** too many layers $\rightarrow$ *oversmoothing*

Model	aes_core	aes_enc	aes_master	apbtoaes128	gcm_aes	Avg.
Random Forest ($n=300$)	0.75	0.35	0.33	0.36	0.74	0.51
TabPFN	0.76	0.39	0.44	0.29	0.74	0.52
TabICL	0.77	0.38	0.30	0.30	0.77	0.50
GraphSAGE	0.78	0.61	0.44	0.27	0.76	0.57

Table: PR-AUC of baseline methods and selected GNN architecture for AES families under the LODO setting. Bold indicates the best average performance.

Model	aes_core	aes_enc	aes_master	apbtoaes128	gcm_aes	Avg.
GIN	0.43	0.53	0.19	0.17	0.38	0.34
GCN	0.48	0.58	0.41	0.19	0.51	0.43
GAT (head=8)	0.73	0.51	0.30	0.21	0.68	0.49
GaAN (head=8)	0.76	0.44	0.36	0.26	0.63	0.49
GraphSAGE	0.76	0.51	0.38	0.29	0.78	0.54

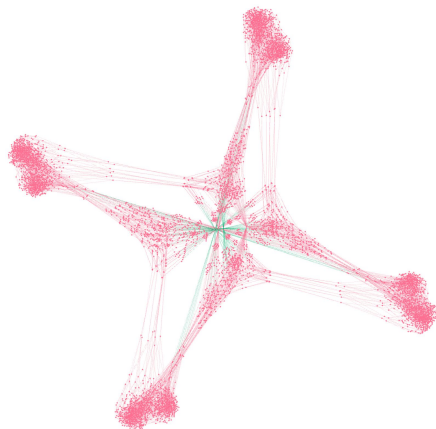
Table: PR-AUC comparison of GNN architectures on the AES LODO setup. All models use 4 layers, 256 hidden dimensions, and weighted cross-entropy loss. Each column represents the held-out design.

- PR-AUC:

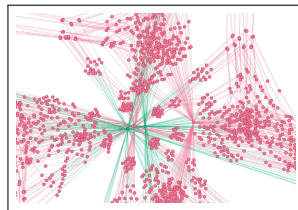
- 1 GraphSAGE = 0.43
- 2 Bi-directed GraphSAGE = 0.45
- 3 JK GraphSAGE = 0.43

Model	aes_core	aes_enc	aes_master	apbtoaes128	gcm_aes	Avg.
<i>Layers</i>						
GraphSAGE (2L, 256HD)	0.65	0.62	0.33	0.20	0.71	0.50
GraphSAGE (4L, 256HD)	0.77	0.53	0.46	0.29	0.76	0.56
GraphSAGE (6L, 256HD)	0.78	0.61	0.44	0.27	0.76	0.57
GraphSAGE (8L, 256HD)	0.75	0.57	0.36	0.30	0.75	0.55
<i>Hidden dimension</i>						
GraphSAGE (6L, 128HD)	0.77	0.52	0.43	0.16	0.44	0.46
GraphSAGE (6L, 256HD)	0.78	0.61	0.44	0.27	0.76	0.57
GraphSAGE (6L, 512HD)	0.79	0.56	0.43	0.30	0.76	0.57

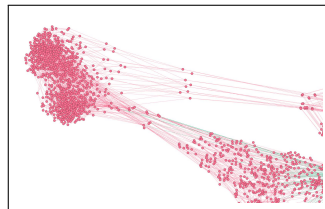
Table: Evaluation of depth and hidden dimension size on model performance for the GraphSAGE model. We use GraphSAGE with mean aggregation and projection layers.



(a) Full graph predictions.

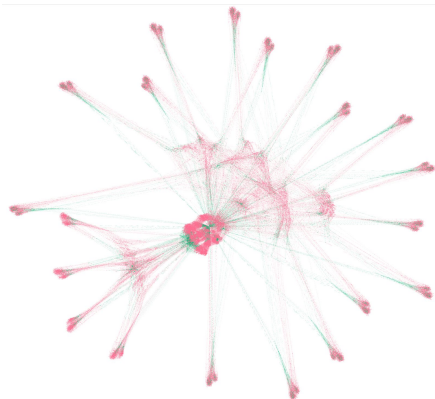


(b) Zoomed-in center region.



(c) Zoomed-in corner region.

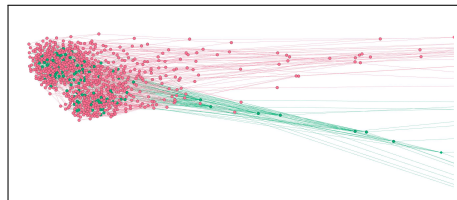
Figure: Design: aes_key_expand_128 (osu035). Pink: correct, Green: incorrect predictions. PR-AUC = 0.97, Baseline = 0.07.



(a) Full graph predictions.



(b) Zoomed-in center region.



(c) Zoomed-in corner region.

Figure: Design: aes_inv_cipher_top (osu035). Pink: correct, Green: incorrect predictions. PR-AUC = 0.76, Baseline = 0.30.

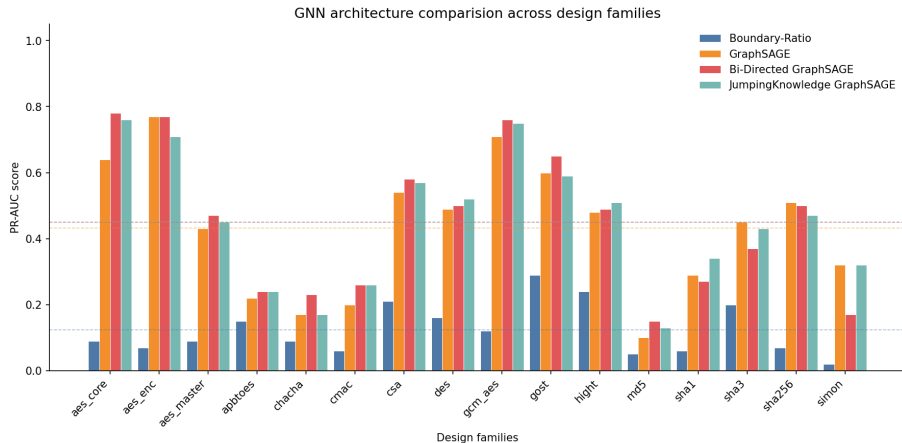
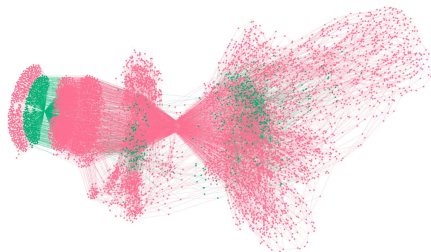
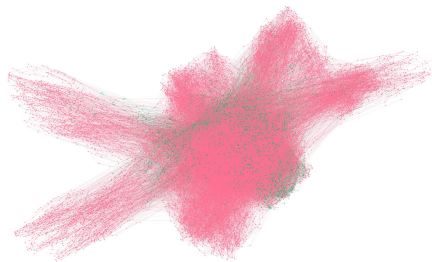


Figure: Performance reported across all the graphs for three architectures: standard (0.43), Bi-directed (0.45), and Jumping Knowledge GraphSAGE (0.43) (16 families).



(a) Family: sha1-master, Design: sha1-core (gsc145nm), PR-AUC: 0.21, Baseline = 0.07.

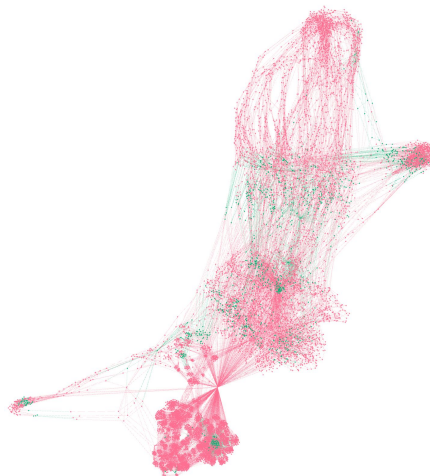


(b) Family: chacha-master, Design: chacha (osu035), PR-AUC: 0.06, Baseline = 0.04.

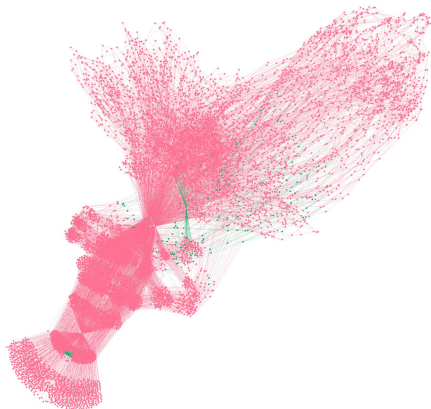
Figure: Pink: Correct predictions, Green: Incorrect predictions.



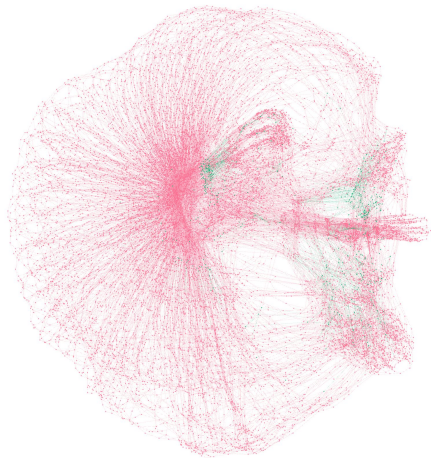
(a) Family: apbtoaes128_latest,
Design: aes_core (nangate), PR-AUC:
0.28, Baseline = 0.18.



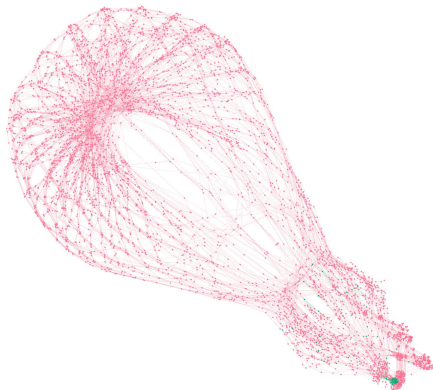
(b) Family: md5-core, Design: md5_ctl
(osu035), PR-AUC: 0.13, Baseline =
0.06.



(a) Family: sha1_master, Design: sha1_core (osu035), PR-AUC: 0.95, Baseline = 0.07.

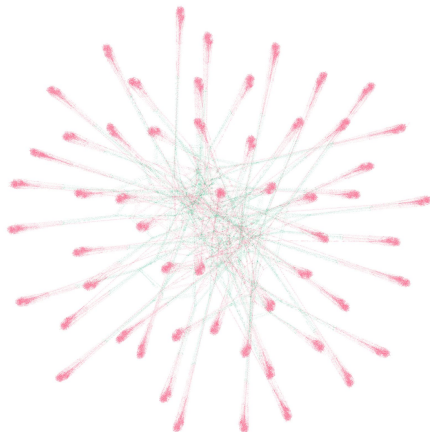


(b) Family: mips32r1_latest, Design: ALU (gsc145nm), PR-AUC: 0.87, Baseline = 0.08.

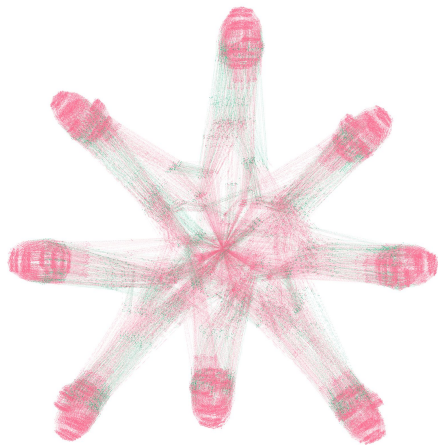


(a) Family:

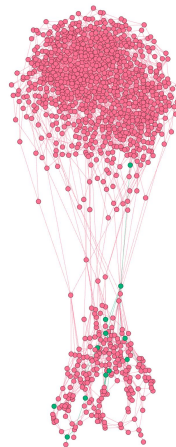
`orsoc_graphics_accelerator`, Design:
`gfx_renderer (osu035)`, PR-AUC: 0.94,
Baseline = 0.09.



(b) Family: `csa_latest`, Design:
`block_decypher (gsc145nm)`, PR-AUC:
0.83, Baseline = 0.06.



(a) Family: `gost28147-89_latest`,
Design: `pipelined_ecb_encrypt`
(`nangate`), PR-AUC: 0.95, Baseline =
0.44.



(b) Family: `r_5_3_gf256_latest`, Design:
`SYN2ERR` (`osu035`), PR-AUC: 0.95,
Baseline = 0.07.

Q2: Does graph structure affect performance?

Structural feature comparison for boundary and internal nodes for successful vs failed designs

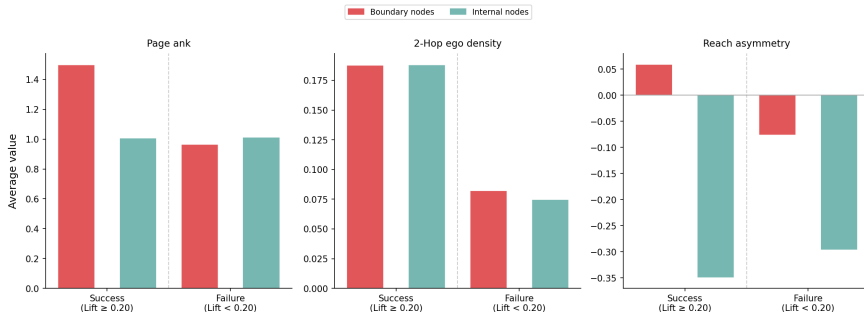
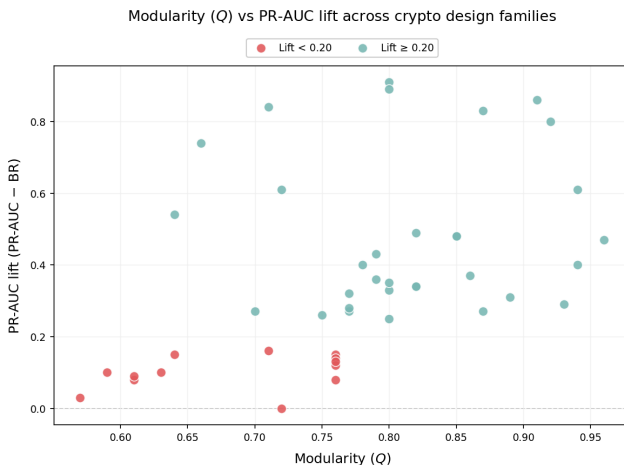


Figure: Average structural features for boundary vs internal nodes showing structural pattern contrasts (successful and failed design).

Q2: Does graph structure affect performance?



- Higher modularity (Q) correlates with better performance ($\rho = 0.549, p < 0.001$)

Q2: Structural features correlation table

Feature	Node type	Success (mean)	Failure (mean)	ρ	p
Modularity (Q)	—	—	—	0.549	<0.001
PageRank	Boundary	1.499	0.965	0.400	0.007
	Internal	1.007	1.014	-0.120	0.434
2-hop ego density	Boundary	0.188	0.082	0.377	0.011
	Internal	0.188	0.075	0.356	0.016
Reach asymmetry	Boundary	+0.059	-0.076	0.455	0.002
	Internal	-0.350	-0.297	-0.235	0.120

Table: Mean structural feature values for successful (lift ≥ 0.20) and failed (lift < 0.20) designs, along with Spearman's rank correlation (ρ) between each feature and PR-AUC lift across 45 crypto designs. **Bold** indicates $p < 0.05$.

Q2: Structural features failure table

Condition	Total	Failures	Failure rate
Overall	45	13	28.9% [18%–43%]
<i>Single conditions</i>			
Low modularity ($Q < 0.787$)	22	13	59.1% [39%–77%]
Negative boundary reach asymmetry (< 0)	22	10	45.5% [27%–65%]
Below-average boundary PageRank (< 1.0)	16	9	56.2% [33%–77%]
<i>Combined conditions (AND)</i>			
Low modularity + negative boundary reach asym.	14	10	71.4% [45%–88%]
Low modularity + low boundary page rank	14	9	64.3% [39%–84%]
Negative reach asymmetry + low page rank	10	7	70.0% [40%–89%]
<i>Combined conditions (OR)</i>			
All three conditions met	8	7	87.5% [53%–98%]
Any one condition met	30	13	43.3% [27%–61%]
None of three conditions met	15	0	0.0% [0%–20%]

Table: The threshold features are chosen based on interpretability (Page rank = 1.0, reach asymmetry = 0.0, Modularity = 0.787 (median)). Failure: PR-AUC lift < 0.20 .

- 2-hop ego density correlates with PR-AUC lift ($\rho = 0.377$, $p = 0.011$)
- However, we excluded it as a failure predictor (at median = 0.110):
 - Flagged 6 extra graphs as risky (all succeeded)
 - Did not identify any additional failures beyond the 13 already captured by modularity, reach asymmetry, and PageRank

Graph	PageRank		2-hop ego		Reach Asym		Mod (Q)	BR	PR-AUC
	B	I	B	I	B	I			
sha1_core (gscl45nm)	0.77	1.09	0.02	0.10	-0.38	-0.32	0.62	0.07	0.21
sha1_core (nangate)	0.90	1.06	0.06	0.10	-0.36	-0.29	0.62	0.01	0.11
sha1_core (osu035)	0.79	1.08	0.36	0.09	0.33	-0.30	0.66	0.07	0.95

Table: Structural characteristics of sha1-master across three libraries. Bold marks features showing failure patterns. B = boundary nodes, I = internal nodes. **Pink** = PR-AUC lift < 0.20.

- LODO trained on 2 libraries, tested on unseen 3rd
- Performance stays stable
- Model does not learn library-specific patterns, but structural features only

Family	Library	w/ library	w/o library
hight_latest	osu035	0.54	0.56
hight_latest	nangate	0.63	0.61
hight_latest	gsc145nm	0.44	0.48

Table: Design: hight_latest (50 family LODO).

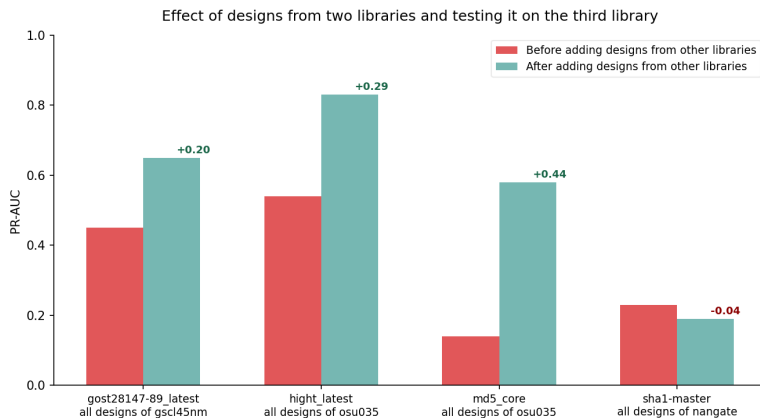


Figure: Model performance before and after modified LODO setup.

Feature ablations across design families (Model: Bi-Directed GraphSAGE)

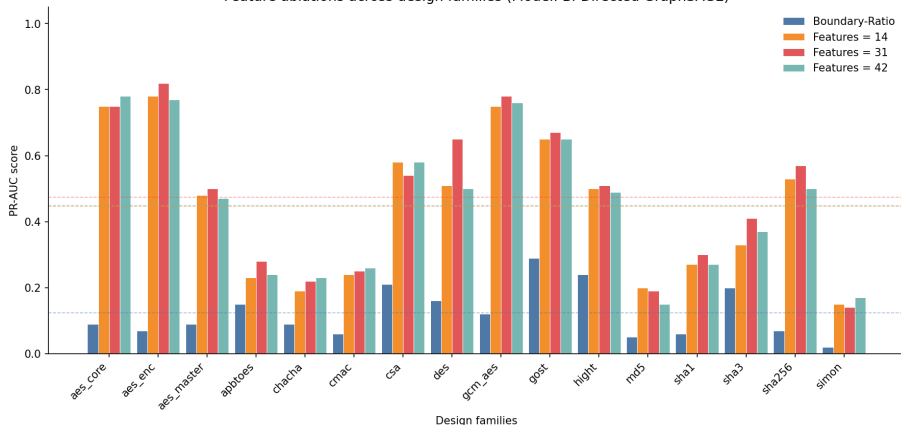


Figure: Feature ablation for GNN. The PR-AUC achieved by the Bi-directed GNN for 14, 31, and 42 feature sets is 0.45, 0.47, and 0.45.

Feature ablations across design families (Model: Random Forest)

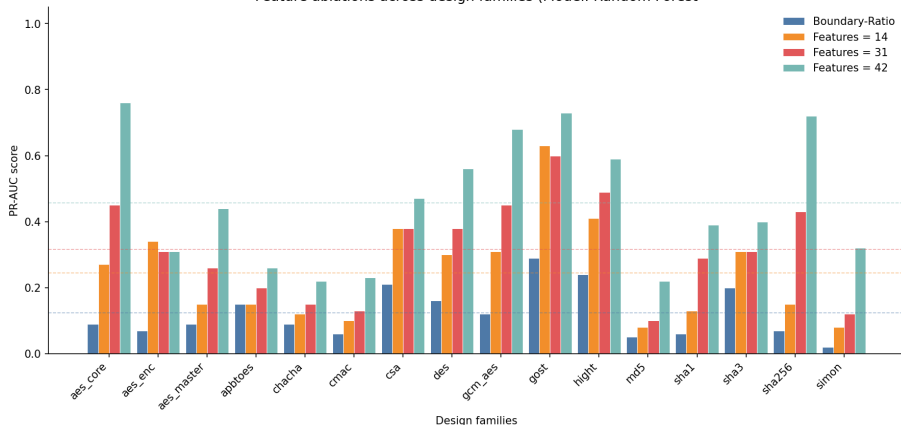


Figure: Feature ablation for random forest ($n = 300$). The PR-AUC achieved at 14, 31, and 42 feature sets is 0.26, 0.32, and 0.44, respectively.

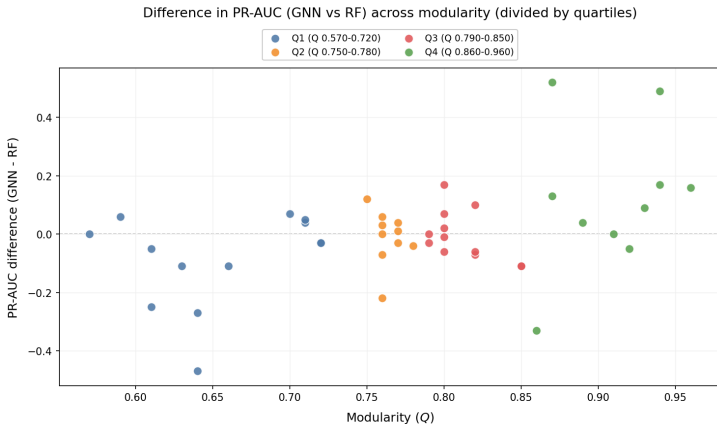


Figure: PR-AUC difference in GNN vs RF performance across the Crypto designs. We divide the set by quartiles.

Feature	GNN		RF	
	ρ	p	ρ	p
Modularity (Q)	0.55	<0.001	0.24	0.111
PageRank (boundary)	0.40	0.006	0.12	0.437
2-hop ego density (boundary)	0.38	0.011	0.32	0.031
Reach asymmetry (boundary)	0.46	0.002	0.47	0.001

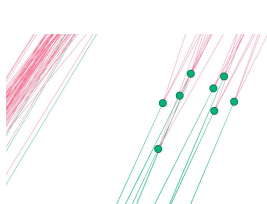
Table: Correlations between structural features and PR-AUC lift values. **Bold** indicates $p < 0.05$.

Subcircuit	ACC	F1	Design
aes_sbox (seen)	73.89%	0.80	aes_inv_cipher_top
aes_inv_sbox (unseen)	00.00%	0.00	aes_inv_cipher_top
aes_decipher_block (unseen)	00.00%	0.00	aes_decipher_block
aes_inv_sbox (unseen)	00.00%	0.00	aes_decipher_block

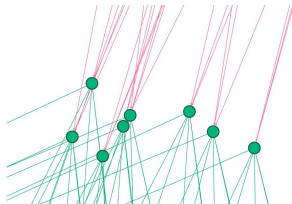
Table: GNN-RE results (osu035) on AES family LODO setup.

- Predicted distribution for aes_decipher_block: aes_key_expand_128 (2571), aes_sbox (2344), gcm_aes_v0 (1573) ... (all unrelated classes).
- GNN-RE performs well on subcircuit types seen during training
- Performance drops to zero for unseen subcircuit types and predicts random labels
- Requires a sufficiently large and diverse dataset for every possible subcircuit with retraining (difficult to construct in practice)

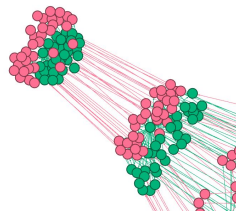
- **More diversity:** improves generalization to unseen designs
 - Example: `aes_enc` family, PR-AUC: 0.56 $\rightarrow$ 0.77 with more diverse training
 - Wider range of structures $\rightarrow$ model learns general representations
- **Targeted training:** for strong performance on specific designs, effective to include structurally similar examples
 - Example: `aes_core` family, PR-AUC: 0.78 $\rightarrow$ 0.62 as diversity increases
 - Controlled variation rather than maximizing diversity



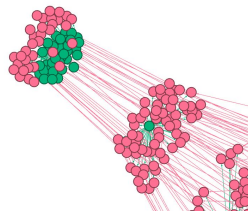
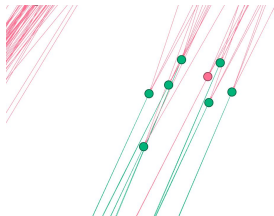
aes-core



gcm-aes



hight_latest



Row 1: Ground truth, Row 2: Predicted. Pink = non-boundary, Green = boundary.